Bases de données avancées & Big Data

Dr FERRAHI Ibtissem

4

Introduction au Big Data

Les règles de CODD

- **Règle 0 : Toutes les fonctionnalités du SGBDR doivent être disponibles** à travers le modèle relationnel et le langage d'interrogation.
- **Règle 1 :** Toutes les données sont représentées par des valeurs présentes dans des **colonnes** et des **lignes** de tables.
- **Règle 3 :** Une cellule peut ne pas contenir de valeur, ou exprimer que la valeur est inconnue, à l'aide du marqueur **NULL**. Il s'agit d'un indicateur spécial, distinct de toute valeur et traité de façon particulière.
- **Règle 5 :** Le SGBDR doit implémenter un langage relationnel qui supporte des fonctionnalités de manipulation des données et des métadonnées, de définition de contraintes de sécurité et la gestion des transactions.

Les règles de CODD

- **Règle 10 : Indépendance d'intégrité :** les contraintes d'intégrité doivent être indépendantes des programmes clients et doivent être stockées dans le catalogue du SGBDR. On doit pouvoir modifier ces contraintes sans affecter les programmes clients.
- **Règle 11 : Indépendance de distribution :** la distribution ou le partitionnement des données ne doivent avoir aucun impact sur les programmes clients.
- **Règle 12 : Règle de non-subversion :** aucune interface de bas niveau ne doit permettre de contourner les règles édictées. Dans les faits, cette règle implique qu'il n'est possible d'interroger et de manipuler le SGBDR qu'à travers son langage relationnel.

Les bases de données relationnelles

- **Les bases de données relationnelles sont basées sur trois éléments:**
  - Structure de données
  - Des contraintes
  - Des opérations
- **Limites**
  - La distribution des données sur plusieurs serveurs devient impossible dans le cas ou il y a un très grand nombre de relations
  - Le temps de vérification des contraintes d'intégrités est très élevé par rapport au simple stockage sans contraintes
  - Les données doivent suivre la structure d'un schéma défini préalablement donc, il n y 'pas de possibilité de sauvegarder des données semi structurées

Limites des SGBDR

- **Limites**
- Jointures entre tables pour des requêtes complexes impliquant plusieurs entités
- Intégrité référentielle permettant d'assurer la validité des liens entre entités
- Données liées entre elles placées sur le même nœud du serveur
- Si nombre de liens important, il est difficile de placer les données sur des nœuds différents

→ *SQL : schéma fermé et performances faibles sur de gros volumes*

Limites des SGBDR

- SGBD R généralement transactionnels
  - **Contraintes ACID :**
    - **A**tomicité des transactions,
    - **C**ohérence des données,
    - **I**solation des modifications,
    - **D**urabilité en cas de perte d'un disque

Propriétés ACID

- **Propriétés ACID des transactions des BDR**
  - **Atomicité :** Soit toutes les instructions sont exécutées, soit aucune
  - **Cohérence :** toute transaction amène la BD d'un état valide à un autre (renforcée par les contraintes d'intégrité et les clés étrangères)
  - **Isolation :** Même si plusieurs transactions peuvent être exécutées par un ou plusieurs utilisateurs simultanément, une transaction ne devrait pas voir les effets des autres transactions concurrentes
  - **Durabilité :** Une fois la transaction enregistrée dans la base (commit), ces changements sont persistants.
- Toutes les BDR supportent les transactions ACID

Limites des SGBDR

- SGBD R généralement transactionnels
  - **Contraintes ACID :**
    - **A**tomicité des transactions,
    - **C**ohérence des données,
    - **I**solation des modifications,
    - **D**urabilité en cas de perte d'un disque
- Coût considérable pour les contextes fortement distribués
- ACID difficile à maintenir à l'échelle du système distribué tout en maintenant des performances correctes

→ La plupart des SGBD « NoSQL » relâchent les contraintes ACID, ou même ne proposent pas de gestion de transactions

Contexte

- **Traitement des données volumineuse**
  - Tendance à tous numériser et sauvegarder pour en tirer des connaissances
  - Numérisation des administrations : APC (état civil), Justice (Casier judiciaire), etc.
  - **Capteurs :** température, tension, vent, humidité, etc.
  - Smartphones
  - Réseaux sociaux, web, etc.
  - Données de localisation : GPS, parcours, chemins, etc.
  - Données vitales : battements de cœur, respiration, effort, etc.
  - Données médicales : rapports, radio, médicaments, etc.
  - Objets connectés grâce aux protocoles NFC, Bluetooth, Zigbee : Montres, machines, équipements, etc.

Contexte

- **Essor des grandes plateformes et application web**
  - Google, Facebook, Amazon
- **Volume important de données à gérer par ces applications →Nécessité de distribution de données et leur traitement sur de nombreux serveurs** « Data Centre »
- **Ces données sont souvent associées à des objets Complexes et Hétérogènes**
  - Texte, Photo, son, compte …

→ Limites des SGBD traditionnels (relationnels et transactionnels) basés sur SQL

→Nécessité de nouvelle approches de stockage et de gestion de données

Nouveaux besoins en gestion des données

D'où nouvelles approches de stockage et de gestion des données :

1. Permettant une meilleure scalabilité (passage à l'échelle) dans des contextes fortement distribués
2. Permettant une gestion d'objets complexes et hétérogènes sans avoir à déclarer au préalable l'ensemble des champs représentant un objet
3. Regroupées derrière le terme NoSQL (proposé par Carl Strozzi), ne se substituant pas aux SGBD Relationnels mais les complètent en comblant leurs faiblesses (Not Only SQL)

Nouveaux besoins en gestion des données

- Les nouvelles approches de stockage et de gestion doivent:
  - **L'extensibilité horizontale:** Assurer une meilleur scalabilité dans les contexte fortement distribuées
  - Permettre la gestion des objet complexes et hétérogènes
  - Utiliser les BDs NoSQL
  - **Gestion distribuée des données**
    - **Distribuer les traitement**
      - Utiliser plusieurs machines afin d'absorber les charges très importante
    - **Distribuées les données**
      - Utiliser plusieurs serveur afin de stocker les données

NoSQL: Not only SQL

- Inventé en 2009 pour les bases de données distribuées
- Terme vague et incorrect : certains sont des variantes du SQL

- Big Data : ensembles de données très volumineux
  → *outils informatiques classiques dépassés*

- Gestion et traitement des volumes de données
  → *nouveau défi de l'informatique*

NoSQL: Not only SQL

- **À quoi ressemble une BD NoSQL**
- Un SGBD NoSQL n'est pas structuré en tables
  - L'élément de base n'est pas un tuple mais dépend du type de BD NoSQL ;

- Un langage de requête non uniformisé, propre à chaque BD
- Une dénormalisation des données où certains enregistrements sont en partie ou entièrement dupliqués ;

- Type de base de données NoSQL à choisir en fonction de l'usage souhaité
  - Plusieurs types de BD NoSQL existent
  - Voldemort, Cassandra, Dynomite, Hbase, Hypertable, CouchDB et MongoDB**le une BD NOSQL**

Avantages du NOSQL

- **Adaptation des BD NOSQL au BigData**
  - **Vitesse (Vélocité) :** beaucoup de données qui arrivent rapidement, à partir de plusieurs sources
  - **Variété :** données structurées, semi-structurées ou non-structurées
  - **Volume:** données très nombreuses (To et Po)
  - **Complexité:** données stockées et gérées à plusieurs endroits et data centers.
- **Capacités transactionnelles modernes**
  - Nouvelles définition du principe de transaction
  - Utilisation des propriétés BASE au lieu des propriétés ACID

Avantages du NOSQL

- **Adaptation des BD NOSQL au BigData**
  - **Vitesse (Velocity)**
    - **Le système doit être disponible**
  - **Variété (Variety) :**
    - **Système doit être flexible**
  - **Volume (Volume) :**
    - **Le système doit être scalable**
  - **Complexité (Complexity) :** données stockées et gérées à plusieurs endroits et data centers.

Avantages du NOSQL

- **Disponibilité continue des données**
  - Le manque de disponibilité peut être fatal pour une entreprise
  - Les BD NOSQL utilisent une architecture hautement distribuée :
    - pas de SPOF (single point of failure ou point unique de défaillance: le système est dépendant de ce point dont la panne entraine l'arrêt du système informatique)
  - Redondance des données et traitements : tolérance aux fautes

→D'où : disponibilité continue à travers les data centers, et le cloud

Avantages du NOSQL

- **Indépendance de l'emplacement**
  - Possibilité de consulter et modifier une BD sans savoir où est-ce que ces opérations ont réellement lieu
  - Toute fonctionnalité d'écriture propagée à partir d'un emplacement, pour être disponible aux utilisateurs à partir d'autres sites
  - Difficile à appliquer aux BDR, surtout pour l'écriture

Avantages du NOSQL

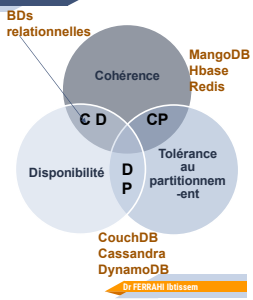
- **Modèles de données flexibles (avantage majeur)**
  - Dans le modèle relationnel, les relations entre les tables sont prédéfinies, fixes et organisées dans un schéma strict et uniforme
    - Problèmes d'évolutivité et de performance en gérant de grands volumes de données
  - Les BD NOSQL peuvent accepter tout type de données (structurées, semi-structurées ou non-structurées) plus facilement
- **Business Intelligence et Analyse**
  - Capacité des bases NOSQL à exploiter les données collectées pour dériver des connaissances
  - Extraction d'informations décisionnelles à partir d'un grand volume de données, difficile à obtenir avec des bases relationnelles

Théorème CAP

- **Limites des propriétés ACID :**
  - Données de plus en plus volumineuses + Besoin de systèmes évolutifs
  - Besoin de BD distribuées sur le réseau, pour une évolutivité horizontale
- **Propriétés CAP : Consistency, Availability, Partition tolerance**
  - Destiné à évaluer les systèmes de stockage distribués
- **Théorème CAP (Brewer, 2000):**
  - « Il est impossible de satisfaire les trois propriétés CAP en même temps »

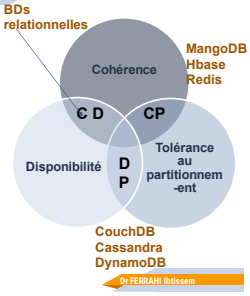
THÉORÈME CAP

- **Propriété fondamentales des systèmes distribuées**
  - **Cohérence ou consistance:** Tous les nœuds voient les mêmes données ou même moment
    - pour avoir une cohérence de 100%, les nœuds doivent communiquer
    - plus il y a de répliques et plus les performances du système sont mauvaises



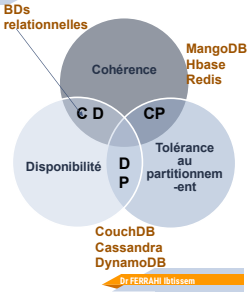
THÉORÈME CAP

- **Propriété fondamentales des systèmes distribuées**
  - **Disponibilité:** la perte de nœuds n'empêche pas les survivants de continuer à fonctionner correctement, les données restent accessibles
    - une réplication de la BD sur un autre serveur assure une disponibilité de 100%
    - les répliques aident à équilibrer la charge d'opérations, notamment en lecture
  - **Tolérance ou partitionnement:** le système étant partitionnée, aucune panne moins importante qu'une coupure totale du réseau ne doit empêcher de répondre correctement



THÉORÈME CAP

- **Propriété fondamentales des systèmes distribuées**
  - **Théorème CAP (Brewer, 2000) : impossible d'avoir ces 3 propriétés en même temps**
  - **Les SGBDR** assurent la cohérence et la disponibilité
  - **Les SGBD NoSQL** sont des systèmes CPs ou DPs



THÉORÈME CAP

- **C - Consistance (Coherence) :** Un système S est dit consistant si on peut garantir qu'une fois qu'on y enregistre un état (disons "x = y"), il donnera le même état à chaque opération suivante, jusqu'à ce que cet état soit explicitement changé par quelque chose en dehors de S

  - **Exemple 1:**

Une instance de la BD est automatiquement pleinement consistante puisqu'il n'y a qu'un seul nœud qui maintient l'état.

THÉORÈME CAP

- **C - Consistance (Coherence) :** Un système S est dit consistant si on peut garantir qu'une fois qu'on y enregistre un état (disons "x = y"), il donnera le même état à chaque opération suivante, jusqu'à ce que cet état soit explicitement changé par quelque chose en dehors de S

  - **Exemple 2 :**

Si 2 serveurs de BD sont impliqués, et si le système est conçu de telle sorte que toutes les clés de "a" à "m" sont conservées sur le serveur 1, et les clés "n" à "z" sont conservées sur le serveur 2, alors le système peut encore facilement garantir la cohérence

THÉORÈME CAP

- **C - Consistance (Coherence) :** Un système S est dit consistant si on peut garantir qu'une fois qu'on y enregistre un état (disons "x = y"), il donnera le même état à chaque opération suivante, jusqu'à ce que cet état soit explicitement changé par quelque chose en dehors de S

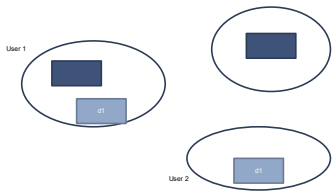
  - **Exemple 3 :**

Supposons 2 BD avec des répliques. Si une des BD fait une opération d'insertion de ligne, cette opération doit être aussi faite (commited) dans la seconde BD avant que l'opération soit considérée comme complète.

  - Pour avoir une cohérence de 100% dans un tel environnement répliqué, les nœuds doivent communiquer
  - Plus le nb de répliques est grand plus les performances d'un tel système sont mauvaises.

THÉORÈME CAP

- **C - Consistance (Coherence) :** Un système S est dit consistant si on peut garantir qu'une fois qu'on y enregistre un état (disons "x = y"), il donnera le même état à chaque opération suivante, jusqu'à ce que cet état soit explicitement changé par quelque chose en dehors de S



Dans un système consistant :
La données d1 doit avoir la même valeur dans chaque copies
- Si on veut changé l'une des valeur , la deuxième copie doit être verrouillée jusqu'à ce que elle reçoit la nouvelle valeur
→ problème de disponibilité

Dans un système disponible :
Communication entre les copie

Théorème de CAP

A - Disponibilité (Availability): Les BD dans Exemple 1 ou Exemple 2 ne sont pas fortement disponibles

- **dans Exemple 1 :** si le nœud tombe en panne, on perd 100% des données
- **dans Exemple 2 :** si le nœud tombe en panne, on perd 50% des données
- **dans Exemple 3 :** une simple réplication de la BD sur un autre serveur assure une disponibilité de 100%.

- Augmenter le nb de nœuds avec des copies des données (répliques) augmente directement la disponibilité du système, en le protégeant contre les défaillances matérielles
- Les répliques aussi aident à équilibrer la charge d'opérations concurrentes, notamment en lecture.

Théorème de CAP

P - Résistance au partitionnement (Partition-tolerance):

- Supposons que soit assuré par réplication «consistance » et «disponibilité ». Dans le cas Exemple 3, supposons 2 serveurs de BD dans 2 Data-Centers différents, et que l'on perde la connexion réseau entre les 2 Data-Centers, faisant que les 2 BD sont incapables de synchroniser leurs états.

- Si on peut gérer les opérations de lecture/écriture sur ces 2 BD, il peut être prouvé que les 2 serveurs ne seront plus consistants.

Théorème de CAP

P - Résistance au partitionnement (Partition-tolerance):

Exemple : une application bancaire gardant à tout moment «l'état de votre compte » est l'exemple parfait du problème des enregistrements inconsistants :

- Si un client retire 500000 DA à Oran, cela doit être immédiatement répercuté à Alger, afin que le système sache exactement combien il peut retirer à tout moment
- Si le système ne parvient pas à le faire, cela pourrait mécontenter de nombreux clients
- Si les banques décident que la « consistance » est très importante, et désactivent les opérations d'écriture lors de la panne, alors la «disponibilité» du cluster sera perdu puisque tous les comptes bancaires dans les 2 villes seront désormais gelés jusqu'à ce que le réseau soit de nouveau opérationnel.

Théorème de CAP

Théorème de CAP (Brewer, 2000) :

Dans un système distribué, il est impossible d'obtenir ces 3 propriétés en même temps, il faut en choisir 2 parmi les 3.

- Les SGBDR assurent les propriétés de Consistance et de Disponibilité (Availability) => système AC
- Les SGBD « NoSQL » sont des systèmes :
  - AP (Disponible et Résistant au partitionnement) ou
  - CP (Cohérent et Résistant au partitionnement)

Théorème CAP

Il est impossible de satisfaire les 3 propriétés CAP en même temps :

- Soit un système distribué. On est en train de modifier une donnée sur le noeud N1 et d'essayer de la lire à partir du nœud N2
- N2 peut retourner la dernière bonne valeur dont il dispose, ce qui viole la Consistance
- N2 attend que la nouvelle valeur lui parvienne. Comme c'est un système distribué, les chances d'un échec de transmission sont assez importantes, ce qui provoquera une attente infinie de N2.

→D'où une violation de la Disponibilité

- Si on veut satisfaire à la fois la consistance et la disponibilité, le système de stockage ne doit pas être partitionné. D'où violation de la Tolérance au partitionnement.

Propriétés BASE

- BASE est plus flexible que ACID, accepte certaines erreurs, dont l'occurrence est assez rare

B Basically A Available **S Soft-State**
E Eventual Consistency

- **Basically Available**
  - Le système garantit la disponibilité, comme définie dans le théorème CAP
- **Soft-State**
  - L'état du système peut changer dans le temps, même sans nouvelles entrées, et ce à cause du principe de consistance éventuelle
- **Eventual Consistency**
  - Les modifications arriveront éventuellement à tous les serveurs, si on leur donne suffisamment de temps

Les BD NoSQL: Récapitulatif

Les BD NoSQL :

- Adoptent une représentation de données non relationnelle
- ne remplacent pas les BD relationnelles mais sont une alternative, un complément apportant des solutions plus intéressantes dans certains contextes
- apportent une plus grande performance dans le contexte des applications Web avec des volumétries de données exponentielle
- utilisent une très forte distribution de ces données et des traitements associés sur de nombreux serveurs
- font un compromis sur le caractère « ACID » des SGBDR pour plus de scalabilité horizontale et d'évolutivité.

L'adoption croissante des bases NoSQL par des grands acteurs du Web (Google, faceBook, ...)
→ multiplication des offres de systèmes NoSQL

Les BD NoSQL: Récapitulatif

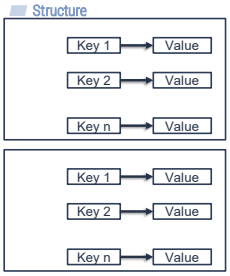
- Pas de schéma pour les données ou schéma dynamique
- Données de structures complexes ou imbriquées
- Données distribuées : partitionnement horizontal des données sur plusieurs nœuds (serveurs) généralement par utilisation d'algorithmes « MapReduce »
- Réplication des données sur plusieurs nœuds
- Privilégient la Disponibilité (A) à la Cohérence (C) (théorème de CAP) : AP (Disponible + Résistant au partitionnement) plutôt que CP (Cohérent + Résistant au partitionnement)

→n'ont en général pas de gestion de transactions

- Mode d'utilisation : peu d'écritures, beaucoup de lectures

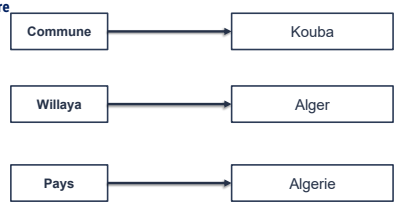
NoSQL orientées Clé/ Valeur

Structure



NoSQL orientées Clé/ Valeur

Structure

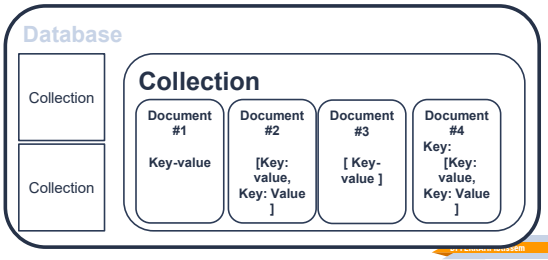


NoSQL orientées colonnes

	Personnel		Contact	
Row key	**Nom**	**Age**	**Téléphone**	**Ville**
0001	Slimani **Timestamp** 19-11-2020 17:00:22	26 **Timestamp** 19-11-2020 17:00:22	05 96 321 47 **Timestamp** 19-11-2020 17:00:22	Boumerdes **Timestamp** 19-11-2020 17:00:22
		35 **Timestamp** 19-11-2021 17:00:22		Rouiba **Timestamp** 19-11-2021 17:00:22
				Bab ezzouar **Timestamp** 19-11-2021 17:00:22
0002	Mezian **Timestamp** 19-11-2020 17:00:22	35 **Timestamp** 19-11-2020 17:00:22	159 236 357 **Timestamp** 19-11-2020 17:00:22	Reghaia **Timestamp** 19-11-2020 17:00:22
0003	Kamil **Timestamp** 19-11-2020 17:00:22	55 **Timestamp** 19-11-2020 17:00:22	548 659 159 **Timestamp** 19-11-2020 17:00:22	Blida **Timestamp** 19-11-2020 17:00:22

NoSQL orientées documents

Structure



NoSQL orientées documents

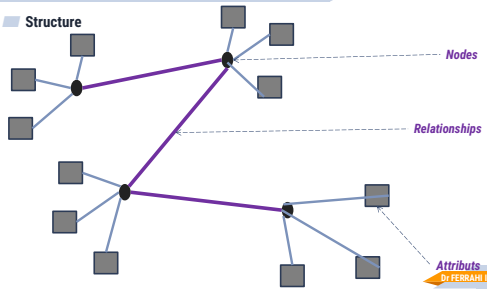
Exemple

```
{
    Nom : '' Ali'' ,
    Age : 35
    Section : '' A ''
    Modules : [ '' Bases de données '', '' Entrepôt de données '' ]

}
```


NoSQL orientées graphe

Structure



Exemple d'application

Nom de l'entreprise	Système NoSQL	Type
Google	bigTable	Column
Linkedin	Voldmort	Key-value
Facebook	Cassandra	Column
Twitter	Hadoop hbase cassandra	Column
Netflix	simpleDB hadoop/Hbase, Cassandra	Column
CERN	CouchBD	Document
Amazon	Dynamo	Key-value

Fondement du NoSQL

- **Sharding :** partitionnement des données sur plusieurs serveurs pour assurer la scalabilité de l'architecture
- **Consistent hashing :** partitionnement horizontal dans lequel nœuds (serveurs) et objets (données) sont associés par une même fonction de hachage
- **Map Reduce :** (dvpé par Google) programmation parallèle sur un grand nb de serveurs : « split, compute, join ».
- Existe dans plusieurs langages (C++, C#, Java, Python)
- **MVCC (Contrôle de Concurrence Multi-Version) :** gère les accès simultanés avec mises-à-jour. Plusieurs versions enregistrées, une seule plus récente, balayage périodique pour supprimer les données obsolètes
- **Vector-Clock** : mises à jours concurrentes datant les données par des vecteurs d'horloge